

Supplementary Material

Bridging the sim2real gap in the table tennis robot with a transformer-based ball states predictor

Yin Bi¹, Christian Conti², Bilan Yang², Alexander Sigrist¹, Peter Dürr¹, Naoya Takahashi¹

¹Sony AI, Zürich, Switzerland

²Sony AI, Tokyo, Japan

I. REAL-WORLD DEPLOYMENT OF PREDICTOR

To evaluate real-world performance, the proposed predictor was integrated into a physical robotic platform for human–robot table tennis rallies. The platform features a custom 8-DoF robot arm (two prismatic and six revolute joints), providing the minimum degrees of freedom required to control racket position, orientation, and shot velocity for competitive play [1], [2]. Raw ball positions and spin measurements from external sensors are provided to the predictor, which generates future ball states from 5–300 ms at 5 ms intervals. These predictions are used as inputs to a control policy governing the robotic arm. The policy is a deep reinforcement learning (RL) policy trained entirely on single shots in simulation using the Soft Actor-Critic (SAC) algorithm [3], which learns to defeat the opponent with a set of desired skills using Universal Value Function Approximators [4]. Training is performed in a custom high-fidelity simulator that models ball flight, ball-table/racket contacts, robot dynamics, delays, and collisions, where parameters are calibrated using real-world ball flight, contact, and robot motion data [1], [5]. During training, the critic receives the ground-truth future ball state from the simulator while the actor receives a predicted ball state from the predictor. A demonstration video showing the integration of our ball-state predictor with a real system for human-robot rallies is included in the **supplementary video**.

II. COMPUTATIONAL COMPLEXITY OF PREDICTOR

In terms of computational complexity, the number of model parameters and floating-point operations (FLOPs) of our proposed predictor are summarized in Table I for different prediction horizons. To enable fast inference, we convert the model into a TensorRT engine [6]. The inference time of the proposed predictor, tested on an NVIDIA RTX 4090, is also reported in Table I. For both FLOPs and inference time measurements, we use a batch size of 1 and full precision (FP32). The measured latency is below 0.5 ms, making it suitable for high-rate perception systems for ball state prediction.

TABLE I: Complexity and running time of the proposed ball states predictor w.r.t. various prediction horizons

Output Length	GFLOPs	Inference time (ms)	Params (#)
60	0.88	0.385	12.6M
45	0.72	0.374	
30	0.43	0.340	

III. BOUNCE EVENT ANALYSIS

In this study, we investigate whether the prediction accuracy degrades significantly across ball–table bounce events. To this end, we computed additional statistics comparing prediction errors before and after table contact. Specifically, we evaluated the prediction errors in ball position, linear velocity, and spin for trajectory segments before and after a bounce. All test samples containing at least one bounce event that were used to evaluate the proposed predictor were included in this analysis, and the mean prediction error is reported. As shown in Fig. 1, the error magnitudes remain at a comparable level before and after the bounce, with no substantial degradation in prediction accuracy following impact. These results indicate that the proposed model captures the dynamics of ball–table contact sufficiently accurately for the downstream control task.

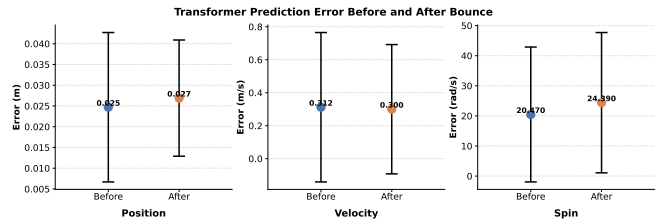


Fig. 1: Comparison of prediction errors before and after ball–table contact with respect to ball position, linear velocity, and spin. The mean position error increased slightly from 2.5 cm before the bounce to 2.7 cm after the bounce, while the mean velocity error changed from 0.312 m/s to 0.300 m/s, and the mean spin error increased from 20.17 rad/s to 24.39 rad/s. These results indicate only a modest change in prediction accuracy across the contact event.

IV. STUDY OF BALL STATE PREDICTOR ARCHITECTURE

In this study, we investigate whether an encoder-only, decoder-only, or encoder-decoder architecture is more suitable for the ball state predictor. For the encoder-only architecture, as illustrated in Fig. 2a, the structured feature matrix produced by the identical blocks is directly fed into a linear projection layer, which maps the learned feature representation to the predicted future ball states. For the decoder-only architecture, as illustrated in Fig. 2b, the cross-attention layer is replaced with a self-attention layer, since no encoder-generated feature matrix is available as input to the decoder. Consistent with the implementation described in Section IV-A-2 & 3, all hyperparameters were kept constant, with only the number of identical layers varied. To have the same number of identical layers across models, both the encoder-only and decoder-only models comprise five identical layers each. All architectures are trained using the same training procedure and the same training and evaluation datasets described in Section IV-A-2 & 3. The results shown in Fig. 3 indicate that the encoder-decoder architecture achieves the lowest prediction errors in position, velocity, and spin compared to the other variants.

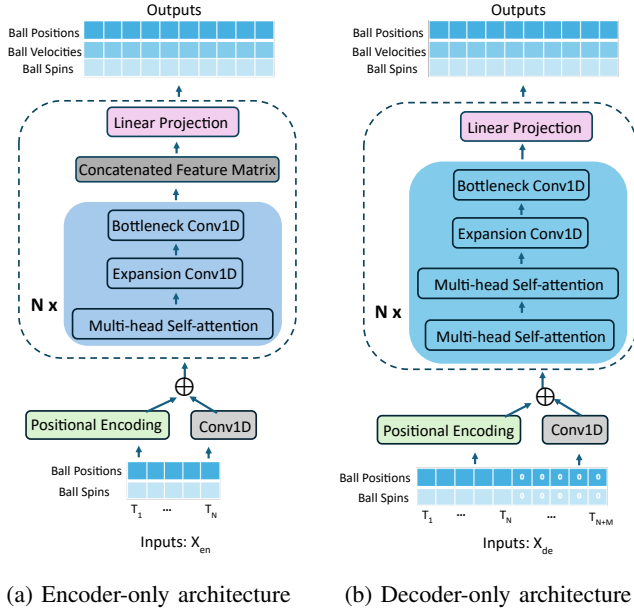


Fig. 2: Overview of the explored encoder-only and decoder-only architectures.

V. INPUT LENGTH FOR BALL STATE PREDICTOR

In this study, we investigate the effect of input sequence length on predictor performance. Specifically, we set the input length N to 12, 10, 8, 6 and 4. The ball-state predictor adopts an encoder-decoder architecture, with the parameters identical to those described in Section IV-A-2 & 3. The training procedure, as well as the training and evaluation datasets, remain consistent with those detailed in Section IV-A-2 & 3. As illustrated in Fig. 4, prediction accuracy degrades as the input length decreases. This finding indicates

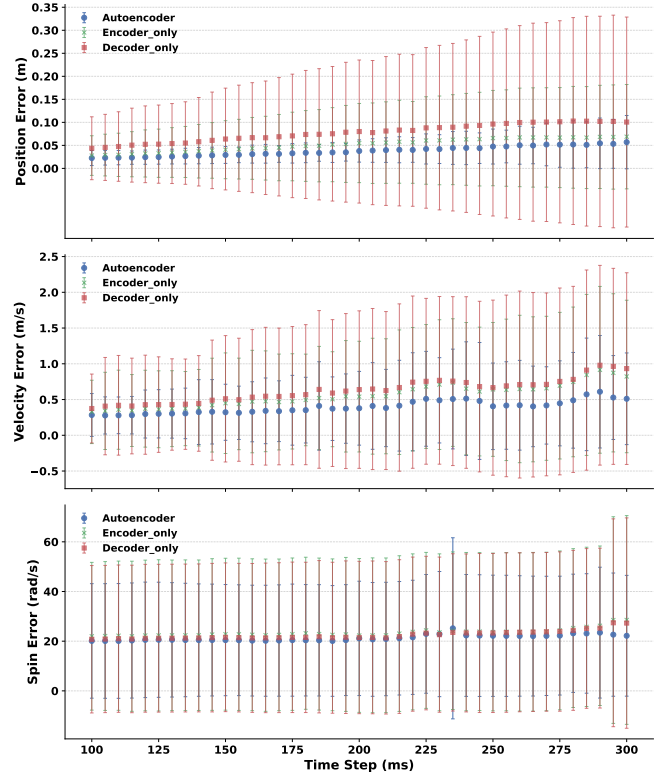


Fig. 3: Accuracy of ball state prediction for position, velocity, and spin. Mean prediction error and standard deviation are reported for prediction horizons from 100 ms to 300 ms.

that achieving higher prediction accuracy requires a longer input sequence, i.e., a greater number of historical ball states.

VI. STUDY ON THE EFFECT OF TRAINING DATASET SIZE

To investigate the amount of real-world data required for training the proposed ball state predictor, we conducted this study by varying the size of the training dataset. Specifically, the predictor was trained using randomly selected subsets containing 20%, 40%, 60%, 80%, and 100% of the original training data. For each subset, the predictor was trained following the same training procedure described in Section IV-A-2 in the manuscript, and evaluated on the same testing dataset used in Section IV-A-3 in the manuscript.

Figure 5 summarizes the prediction performance as the amount of training data is reduced. The results show that the prediction accuracy remains largely unchanged when 80% or 60% of the training data is used. A noticeable degradation is observed when the training set is reduced to 40%. Nevertheless, even the predictor trained with only 20% of the available training data consistently outperforms the baseline predictors evaluated in Section IV-A-3.

These results indicate that, although collecting a large-scale real-world dataset is beneficial, the proposed predictor does not require the full dataset to achieve strong performance. In practice, a substantially smaller dataset is sufficient to obtain prediction accuracy superior to the physics-based

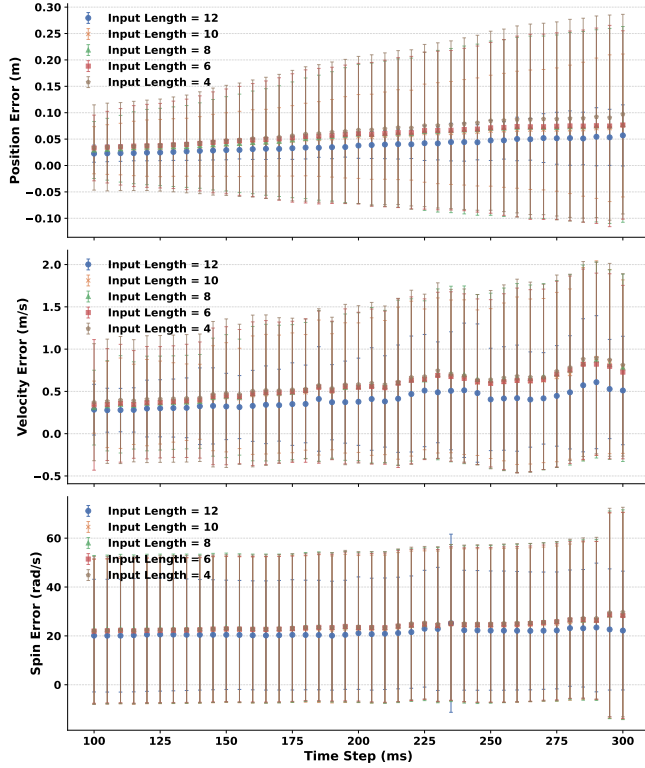


Fig. 4: Accuracy of ball state prediction for position, velocity, and spin w.r.t. input length. Mean prediction error and standard deviation are reported for prediction horizons from 100 ms to 300 ms.

baselines, reducing the burden of real-world data collection.

VII. STUDY ON THE EFFECT OF PREDICTIVE BALL STATES AS POLICY INPUT

To further investigate the role of predicted ball states in the control policy, we conducted an additional study in which the policy was provided with different forms of predictive information. The objective was to evaluate whether explicitly predicting future ball states improves decision-making compared with using only the current estimated state.

The following policy variants were evaluated:

- **Baseline:** The policy receives only the current ball state estimated by the state smoother.
- **Exp. 1 (Fixed-Time Prediction):** The policy receives the current estimated state together with the predicted ball state 50 ms into the future generated by the Transformer predictor.
- **Exp. 2 (Interception-Plane Prediction):** The policy receives the current estimated state together with the predicted ball state whose x -position is closest to the predefined interception plane ($x = -1.2$ m). The table center is the origin, and the robot operates on the negative x -axis side.
- **Exp. 3 (Post-Bounce Prediction):** The policy receives the current estimated state together with the 25 ms length of predicted ball states after the table bounce.

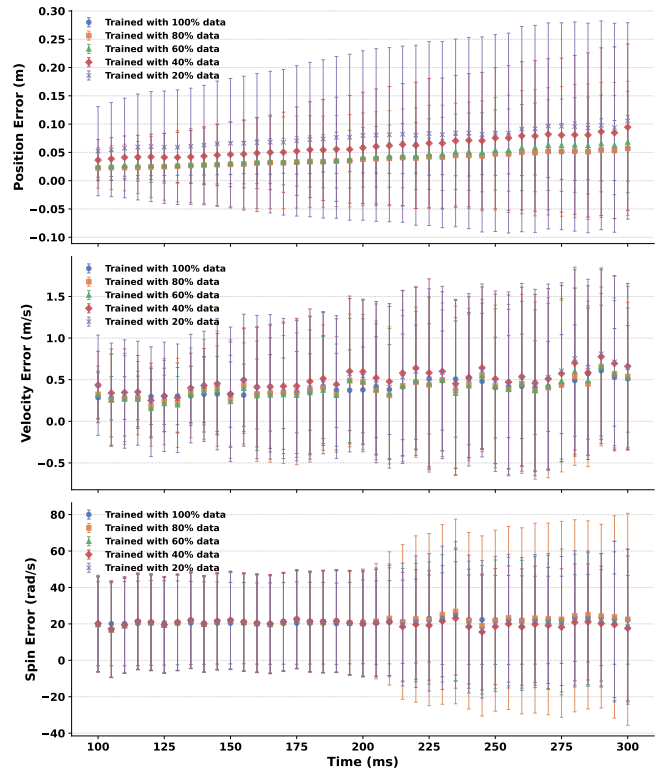


Fig. 5: Effect of the training dataset size on prediction performance. The predictor was trained using randomly selected subsets containing 20%, 40%, 60%, 80%, and 100% of the original training data, while the testing dataset and training procedure remained unchanged. Prediction performance degrades noticeably only when fewer than approximately 40% of the training samples are used. Even with 20% of the training data, the proposed predictor still outperforms the baseline predictors reported in Section IV.

- **Exp. 4 (Prediction Sequence):** The policy receives a sequence of predicted ball states from 5 ms to 300 ms into the future, uniformly sampled every 30 ms.

Note that these experiments were conducted at an early stage of this project using an earlier, more naive version of the control policy. As a result, the policy architecture and implementation differ from those used in the final system reported in the manuscript. Each policy was trained independently using the same training procedure described in the manuscript. To reduce the influence of training randomness, every experiment was repeated three times using different random seeds. The trained policies were then evaluated on 200 test shots, and the average hitting rate and return rate were computed across the three runs.

Table II summarizes the experimental results. Compared with the baseline policy that relies solely on the current estimated state, all policy variants incorporating predictive information achieved substantially higher return rates, with improvements ranging from 10.5 to 17.8 percentage points. The hit rate also remained comparable or improved slightly across all variants, reaching 91.2% when using a single

TABLE II: Effect of different forms of predictive information on policy performance. Each policy was trained with three different random seeds and evaluated on 200 test shots. The last two columns report the average hit rate and return rate across the three runs.

Policy Input	Run 1		Run 2		Run 3		Mean	
	Hits	Returns	Hits	Returns	Hits	Returns	Hit (%)	Return (%)
Current state (Baseline)	177	53	175	45	178	64	88.3	27.0
Baseline + Prediction at 50 ms	186	90	175	82	186	91	91.2	43.8
Baseline + Prediction at $x = -1.2$ m	172	78	181	75	177	72	88.3	37.5
Baseline + 25 ms length of predicted post-bounce states	174	85	174	88	169	96	86.2	44.8
Baseline + Predicted trajectory (5–300 ms, 30 ms interval)	183	90	178	92	174	83	89.2	44.2

prediction 50 ms into the future. These results indicate that providing explicit predictions of future ball states enables the policy to make more informed decisions and substantially improves rally performance.

It is worth noting that these experiments were conducted using an earlier version of the control policy rather than the final policy presented in the main manuscript. Therefore, the absolute performance differs from the results reported in the paper. Nevertheless, the consistent improvements observed across all prediction-based variants demonstrate that future-state prediction serves as an effective intermediate representation for robotic table tennis and contributes positively to policy performance.

VIII. COMPARISON WITH SIM-TO-REAL TRANSFER TECHNIQUES

In this study, we compare SPAD with several commonly used sim-to-real transfer techniques. Discrepancies between simulation and the real system arise from perception noise, sensing delays, computational latency, and actuation uncertainties, which affect transfer performance. A common strategy is to inject these imperfections during training to improve transfer robustness [7]–[10]. Following these works, we implemented the following sim-to-real techniques in our robotic table tennis system.

- **Trick 1 - Perception Noise and Delay:** During training, we injected realistic noise and delays into the ball observations. The noise models were identified from data collected on the real system such that their magnitudes match the performance of our perception pipeline. For the ball position, zero-mean Gaussian noise was added independently to each Cartesian coordinate,

$$\tilde{\mathbf{p}} = \mathbf{p} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma_p), \quad (1)$$

where $\sigma_p = 0.1$ cm, which independently samples Gaussian noise for the three position coordinates. In addition, 1% of position measurements were randomly dropped, and an observation delay uniformly sampled between 9 and 11 ms was introduced.

Similarly, for the ball spin, Gaussian perturbations were added with a magnitude standard deviation of 10 rad/s together with an axis perturbation of 0.15 rad. Furthermore, 5% of spin measurements were randomly dropped, and an observation delay uniformly sampled between 4 and 6 ms was applied.

- **Trick 2 - Computation Latency:** The real robot requires non-negligible computation time for policy inference, trajectory optimization, reset planning, and collision checking. To account for this delay, we introduced a fixed computation latency during training by generating the reinforcement learning observation 5 ms earlier than the actual control execution time. This allows the policy to learn under timing conditions that more closely resemble the real system.
- **Trick 3 - Actuation Model Variance:** The simulator contains many sources of model uncertainty. In this study, we consider uncertainty in the ball–table contact model as a representative example. Specifically, we assumed that the simulated bounce does not perfectly reproduce the real ball dynamics. During training, random perturbations were applied to the ball velocity and spin immediately after each table bounce. The velocity perturbation was uniformly sampled within $[0, 0.5]$ m/s, while the spin perturbation was uniformly sampled within $[0, 5]$ rad/s.

We first trained a baseline policy without any of the above sim-to-real techniques. We then trained separate policies by enabling each technique individually and evaluated their performance in terms of successful hits and successful returns. To evaluate the contribution of SPAD, we replaced the simulator predictor used in the baseline with the real-world predictor while keeping the remaining trained policy unchanged.

The evaluation protocol is identical to that in Section IV-B-2. Specifically, we evaluated all policies using the same real-world dataset consisting of 200 rally shots from university-level players (Stroke-Uni), 200 rally shots from professional-level players (Stroke-Pro), and 100 serves (Receive). All training parameters and evaluation settings were kept identical to those described in Section IV-B-2. The policy input consisted of predicted ball states at future horizons of 170, 175, and 180 ms.

Table III summarizes the results. All investigated sim-to-real techniques improve the baseline policy, indicating that modeling real-world imperfections is beneficial for sim-to-real transfer. Among them, SPAD achieves the largest performance gains on both rally tasks (Stroke-Pro and Stroke-Uni), substantially outperforming the other techniques in terms of both successful hits and returns. On the Receive task, SPAD

TABLE III: Comparison of conventional sim-to-real transfer techniques and SPAD on the real-world evaluation dataset. “Hit” and “Return” denote the numbers of successful hits and successful returns, respectively. The best result for each metric is shown in **bold**.

	Baseline		Trick 1		Trick 2		Trick 3		SPAD	
	Hit	Return	Hit	Return	Hit	Return	Hit	Return	Hit	Return
Stroke-Pro	107	75	135	80	135	85	127	77	163	110
Stroke-Uni	105	70	138	81	151	92	130	89	185	144
Receive	85	68	97	79	95	75	91	68	92	68

provides performance comparable to the conventional sim-to-real techniques, while Trick 1 achieves the highest score.

It should be noted that the policy reported in Section IV-B-2 was trained using all of the above sim-to-real techniques, as well as additional components whose individual effects are not separately evaluated in this study. For example, we use a reward term based on the distance to the table edge, which defines a safety margin around the table boundaries; a return outside this margin is considered unsuccessful. This encourages more conservative behaviors in the real world and helps reduce overshooting caused by model mismatch. In addition, we include a reward shaping term that penalizes the angular velocity of the racket at the moment of ball contact, since the racket-ball interaction model does not accurately capture the effect of angular velocity. Although these techniques generally improve transfer robustness, their combination does not necessarily maximize the evaluation metrics reported here. This suggests that their effects are not simply additive and may interact with each other during policy optimization.

IX. BALL FLIGHT AND BALL-TABLE CONTACT MODELS

The ball flight aerodynamics and the ball-table contact model employed used in this paper are described below.

a) Ball flight aerodynamics.: The aerodynamics of the ball in flight are governed by the drag $\mathbf{f}_d$, Magnus $\mathbf{f}_M$, and gravitational $\mathbf{f}_g$ forces. Given that the ball’s angular velocity $\boldsymbol{\omega}$ is approximately constant over short flight intervals, the flight dynamics can be modeled as:

$$\begin{aligned} m\dot{\mathbf{v}} &= \mathbf{f}_d + \mathbf{f}_M + \mathbf{f}_g \\ &= -\frac{1}{2}c_d\rho_{\text{air}}r^2\pi\|\mathbf{v}\|\mathbf{v} - c_M\rho_{\text{air}}\frac{4}{3}r^3\pi\mathbf{v}\times\boldsymbol{\omega} + m\mathbf{g} \end{aligned} \quad (2)$$

with $\mathbf{v}$ the ball velocity, $\rho_{\text{air}} = 1.204 \text{ kg/m}^3$ (density of dry air at room temperature and standard pressure), $m = 2.7 \times 10^{-3} \text{ kg}$ (ball mass), $r = 0.02 \text{ m}$ (ball radius), $c_d \in [0.32, 0.42]$ (drag coefficient) [11], $\mathbf{g} = [0, 0, -9.81]^T \text{ m/s}^2$ (gravitational acceleration), and $c_M = 0.1 \frac{\|\mathbf{v}\|}{r\|\boldsymbol{\omega}\|} - 0.001$ (Magnus coefficient).

b) Ball-table contact model.: The table contact model [12], which assumes instantaneous point contact, is enhanced to capture some effects of surface contacts on the coefficient of restitution $\varepsilon^{\text{table}}$.

$$\mathbf{v}^+ = \mathbf{C}_{v,v}^{\text{table}}\mathbf{v}^- + \mathbf{C}_{v,\omega}^{\text{table}}\boldsymbol{\omega}^- \quad (3)$$

$$\boldsymbol{\omega}^+ = \mathbf{C}_{\omega,v}^{\text{table}}\mathbf{v}^- + \mathbf{C}_{\omega,\omega}^{\text{table}}\boldsymbol{\omega}^- \quad (4)$$

$$\mathbf{C}_{v,v}^{\text{table}} = \begin{bmatrix} 1-\alpha & 0 & 0 \\ 0 & 1-\alpha & 0 \\ 0 & 0 & -\varepsilon^{\text{table}} \end{bmatrix}, \quad (5)$$

$$\mathbf{C}_{v,\omega}^{\text{table}} = \begin{bmatrix} 0 & \alpha r & 0 \\ -\alpha r & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad (6)$$

$$\mathbf{C}_{\omega,v}^{\text{table}} = \begin{bmatrix} 0 & -\frac{3\alpha}{2r} & 0 \\ \frac{3\alpha}{2r} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad (7)$$

$$\mathbf{C}_{\omega,\omega}^{\text{table}} = \begin{bmatrix} 1-\frac{3\alpha}{2} & 0 & 0 \\ 0 & 1-\frac{3\alpha}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

where $-$ and $+$ are pre- and post- contact quantities respectively, and

$$\alpha = \alpha(\mathbf{v}^-, \boldsymbol{\omega}^-) = \begin{cases} \mu(1 + \varepsilon^{\text{table}}) \frac{v_z^-}{\|\mathbf{v}_T^-\|} & (v_s > 0) \\ \frac{2}{5} & (v_s \leq 0) \end{cases} \quad (9)$$

where the contact type is determined as sliding if $v_s > 0$ and rolling if $v_s \leq 0$, with

$$v_s = v_s(\mathbf{v}^-, \boldsymbol{\omega}^-) = 1 - \frac{5}{2}\mu \left(1 + \varepsilon^{\text{table}}\right) \frac{v_z^-}{\|\mathbf{v}_T^-\|}, \quad (10)$$

$$\mathbf{v}_T = \begin{bmatrix} v_x^- - r\omega_y^- \\ v_y^- + r\omega_x^- \\ 0 \end{bmatrix}. \quad (11)$$

$\varepsilon^{\text{table}}$ and μ are the coefficient of restitution and the dynamic coefficient of friction between ball and table, respectively. ITTF requires $\varepsilon^{\text{table}} = \varepsilon^{\text{table}}(v_z^-) \in [0.88; 0.93]$ for $v_z^- \approx 2.4 \text{ m/s}$ [13] and $\mu = 0.25$.

REFERENCES

- [1] e. Peter Dür, “Outplaying elite table tennis players with an autonomous robot,” *Nature*, vol. 652, pp. 886–891, 4 2026.
- [2] N. Mukai, P. Adodin, S. Heusser, A. Sigrist, D. Grover, G. Torrente, F. Khadivar, T. Kakinuma, and P. D’Áz, “Hardware design for table tennis robot capable of beating professional players,” *arXiv preprint arXiv:2606.26643*, 2026.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. Pmlr, 2018, pp. 1861–1870.
- [4] T. Schaul, D. Horgan, K. Gregor, and D. Silver, “Universal value function approximators,” in *International conference on machine learning*. PMLR, 2015, pp. 1312–1320.
- [5] C. Conti, B. Yang, A. Sigrist, L. Miele, Y. Saraiji, P. Dür, and N. Takahashi, “Physics models for sim-to-real transfer in professional-level robot table tennis,” *arXiv preprint arXiv:2606.28805*, 2026.

- [6] E. Jeong, J. Kim, S. Tan, J. Lee, and S. Ha, "Deep learning inference parallelization on heterogeneous processors with tensorsrt," *IEEE Embedded Systems Letters*, vol. 14, no. 1, pp. 15–18, 2021.
- [7] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [8] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.
- [9] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," *arXiv preprint arXiv:1804.10332*, 2018.
- [10] K. Ota, D. K. Jha, D. Romeres, J. van Baar, K. A. Smith, T. Semitsu, T. Oiki, A. Sullivan, D. Nikovski, and J. B. Tenenbaum, "Towards human-level learning of complex physical puzzles," *CoRR, abs/2011.07193*, 2020.
- [11] T. Miyazaki, W. Sakai, T. Komatsu, N. Takahashi, and R. Himeno, "Lift crisis of a spinning table tennis ball," *European Journal of physics*, vol. 38, no. 2, p. 024001, 2017.
- [12] A. Nakashima, Y. Ogawa, C. Liu, and Y. Hayakawa, "Robotic table tennis based on physical models of aerodynamics and rebounds," in *2011 IEEE International Conference on Robotics and Biomimetics*. IEEE, 2011, pp. 2348–2354.
- [13] The International Table Tennis Federation, "Statutes," 2024. [Online]. Available: https://documents.ittf.sport/sites/default/files/public/2024-02/2024_ITTF_Statutes_clean_version.pdf